Assignment 3: Predictive Analytics

Individual assignment | 100 points | **Deadline: 12 midnight, Monday, 16 November**

Using your data from Assignments 1 and 2, answer one predictive question with two models, improve both of them, and compare the results. Do not repeat data engineering or exploratory analysis.

1. Tasks

Choose **one** problem (regression or classification) with **one** target, and implement **exactly two** techniques from the lectures (for example, Linear Regression and Decision Tree Regression, or Logistic Regression and Decision Tree Classification). Rules:

- Both models use the **same target, predictors, and train/test split**.
- Train both first as **baselines** with default settings and a fixed random seed.
- Report a **reference point** (training-set mean for regression, majority class for classification).
- Do not use ~~automated hyperparameter search (such as grid search), cross validation,~~ neural networks or deep learning.

Use the six steps below as your notebook headings.

Step 1. Define the problem and its relevance

State the predictive question in one sentence and explain its practical relevance (who would use it and what decision it supports).

Step 2. Identify the target and predictors

Name the target and list each predictor with its source table, type, and a one-line justification. Exclude any column that would not be available at prediction time (data leakage).

Step 3. Prepare the data and split it

- **Load** your Assignment 1 data, then **split first** into training and test sets with a fixed random seed.
- **Preprocess** only what your models need (for example, missing values, encoding, scaling), and justify each step. Learn all preprocessing from the training set only, then apply it to the test set.
- **Report** the training and test sizes. For classification, also report the class balance and address imbalance if relevant.

Step 4. Train the two models

Train each model on the training set. Justify each choice in 1–2 sentences, record the key settings in a small table, and show one interpretable output per model (for example, coefficients, feature importances, or the tree).

Step 5. Predict, evaluate, and compare

Predict on the test set and evaluate both models with:

Problem type	Required test-set metrics	Required visualizations
Regression	MAE, RMSE (or MSE), and R^2	Per model: predicted vs. actual plot and residual plot
Classification	Accuracy, precision, recall, F1-score, and confusion matrix	Per model: labeled confusion-matrix plot

Compare training and test scores (overfitting) and each model with the reference point. Interpret the results in plain language, analyze where each model makes its largest errors, and state one strength and one weakness of each model in the context of your dataset.

- **Compare:** one table with the test-set metrics of both models, and one comparison chart.

- **Decide:** which technique performs better and why (at least two pieces of evidence).
- **Limitations:** discuss the main sources of prediction error.

Step 6. Improve both models and ~~explore~~ tune hyperparameters

Improve **each** baseline model using an appropriate technique (for example, regularization or limiting model complexity) and compare the results to determine whether performance improved. ~~by exploring different~~Tune key hyperparameters using grid search with cross-validation~~, and show whether it helped~~.

- **Justify:** identify ~~which~~ a weakness in each baseline, explain how the selected technique addresses it. Specify which hyperparameter to tune ~~will you vary~~?
- **Explore:** for each model, use `GridSearchCV` with cross-validation to test ~~try~~ a small, practical set of values (about 3 to 4) for ~~one or~~ two to three hyperparameters per model. Compare the results in a table and a chart, then select the best setting.
- **Show evidence:** a before-and-after table comparison for each models, including training and test scores). Include one chart comparing baseline and improved model performance~~results~~.
- **Interpret:** explain whether ~~did~~ performance improved, by how much, and why? Does it change your choice of model~~technique you would choose~~? A small or no improvement is acceptable if explained clearly.

2. Deliverables and submission

Submit one executed Jupyter Notebook (.ipynb) to your course GitHub repository, with the six step headings, saved outputs, and brief comments justifying key decisions.

3. Grading rubric

Marks reward sound method and clear interpretation, not model accuracy.

Criterion	Points	Full-credit evidence
Predictive problem and rationale	5	Clear, feasible question with explained practical relevance.
Target and predictor selection	5	Target and predictors listed with source, type, and justification.
Data preparation and split	18	Appropriate, justified and correctly implemented preprocessing; split before preprocessing; no data leakage; same split for both models.
Training the two models	20	Two suitable techniques correctly implemented; choices justified; key settings and interpretable output shown; reference point reported.
Model evaluation and visualization	22	Correct metrics; clear, labeled visualizations; training vs. test scores compared; interpretation with error analysis, strengths, and weaknesses.
Model improvement and Hyperparameter tuning	25	Both models improved using~~with a~~ justified technique; key hyperparameters tuned using cross-validation~~values compared~~, best settings selected and supported by ~~with the best chosen;~~ before/after comparison tables and charts; results clearly and honestly interpreted~~honest interpretation~~.
Code quality and reproducibility	5	Organized, commented notebook that runs top to bottom with saved outputs.

Partial credit: Each criterion is scored by quality: 85–100% complete and well justified; 70–84% minor gaps; 50–69% partial or important errors; below 50% major weaknesses. Extra models or complexity earn no extra credit.